

《数据科学》实验报告

班级	003	姓名	史润	学号	20241220
实验题目	机器学习分析方法				
实验时间	2025-11-8	实验地点	DS1408		
学年学期	2025-2026(1)	实验性质	☐ 验证性 ☐ 设计性 ☒ 综合性		

问题 1. 模拟数据集的构造

学会使用 `sklearn.datasets` 生成三类典型人工数据集，理解各参数含义。

- 1) 使用 `make_classification` 生成一个二分类数据集（200 样本，2 特征，无冗余特征）。
- 2) 使用 `make_regression` 生成一个一元线性回归数据集（200 样本，1 特征，噪声=10）。
- 3) 使用 `make_blobs` 生成一个 2 簇聚类数据集（200 样本，2 特征）。
- 4) 将三组数据分别保存为变量：`X_cls, y_cls`; `X_reg, y_reg`; `X_clust, y_true_clust`。
- 5) 对构造的三组数据进行可视化展示（同一个 `figure` 中的三个子图）

代码：

```
import matplotlib

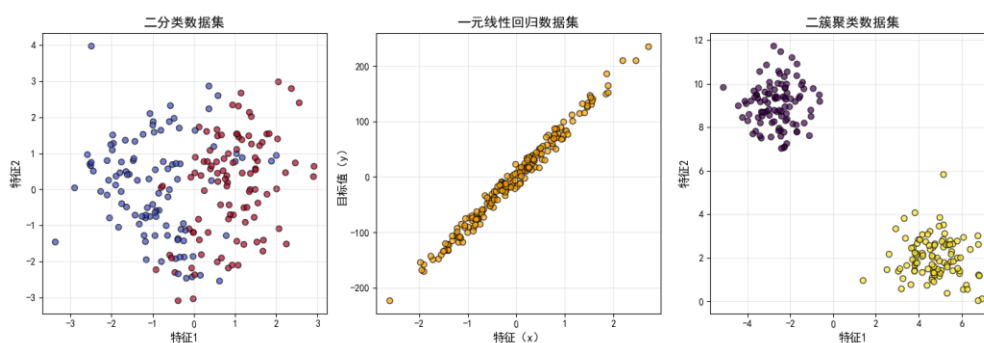
matplotlib.use("TkAgg")
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_regression, make_blobs

# ----- 1. 构造三类模拟数据集 -----
# 1) 二分类数据集：200 样本、2 特征、无冗余特征
X_cls, y_cls = make_classification(
    n_samples=200,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=2,
    random_state=42,
)

# 2) 一元线性回归数据集：200 样本、1 特征、噪声=10
X_reg, y_reg = make_regression(
    n_samples=200,
    n_features=1,
    noise=10,
    random_state=42,
```

```
)
# 3) 二簇聚类数据集: 200 样本、2 特征、2 个簇
X_clust, y_true_clust = make_blobs(
    n_samples=200,
    n_features=2,
    centers=2,
    cluster_std=1.0,
    random_state=42,
)
# ---- 2. 可视化展示 (同一个 figure 的 3 个子图) ----
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
# 子图 1: 二分类数据集
axes[0].scatter(
    X_cls[:, 0], X_cls[:, 1], c=y_cls, cmap="coolwarm", edgecolors="black",
    alpha=0.7
)
axes[0].set_title("二分类数据集", fontsize=14)
axes[0].set_xlabel("特征 1", fontsize=12)
axes[0].set_ylabel("特征 2", fontsize=12)
axes[0].grid(alpha=0.3)
# 子图 2: 一元线性回归数据集
axes[1].scatter(X_reg[:, 0], y_reg, color="orange", edgecolors="black", alp
ha=0.7)
axes[1].set_title("一元线性回归数据集", fontsize=14)
axes[1].set_xlabel("特征 (x)", fontsize=12)
axes[1].set_ylabel("目标值 (y)", fontsize=12)
axes[1].grid(alpha=0.3)
# 子图 3: 二簇聚类数据集
axes[2].scatter(
    X_clust[:, 0],
    X_clust[:, 1],
    c=y_true_clust,
    cmap="viridis",
    edgecolors="black",
    alpha=0.7,
)
axes[2].set_title("二簇聚类数据集", fontsize=14)
axes[2].set_xlabel("特征 1", fontsize=12)
axes[2].set_ylabel("特征 2", fontsize=12)
axes[2].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

结果:



问题 2. 模型评估方法

不依赖 `sklearn.model_selection`, 手动实现留出法和 K 折交叉验证的核心逻辑, 理解其数据划分机制。

- 1) 编写函数 `simple_holdout(X, y, test_size=0.3, random_state=None)`, 返回训练/测试集。
- 2) 编写函数 `simple_kfold(X, y, k=5)`, 返回 K 个 (训练索引, 验证索引) 元组。
- 3) 对分类数据 `X_cls, y_cls` 分别调用两个函数, 验证输出格式正确;

```
# -----2. 模型评估方法 -----
def simple_holdout(X, y, test_size=0.3, random_state=None):
    """
    手动实现留出法 (分层抽样思想, 保证类别分布一致, 适用于分类数据)
    参数:
        X: 特征矩阵 (n_samples, n_features)
        y: 标签向量 (n_samples,)
        test_size: 测试集比例 (默认 0.3)
        random_state: 随机种子 (保证可复现)
    返回:
        X_train, X_test, y_train, y_test: 训练集特征/测试集特征/训练集标签/测试集标签
    """
    if random_state is not None:
        np.random.seed(random_state)
    n_samples = len(X) # 总样本数
    n_test = int(n_samples * test_size) # 测试集样本数 (整数)
    indices = np.arange(n_samples) # 索引数组: [0, 1, 2, ..., n_samples-1]
    np.random.shuffle(indices) # 随机打乱索引
    test_indices = indices[:n_test] # 前 n_test 个索引作为测试集
    train_indices = indices[n_test:] # 剩余索引作为训练集
    X_train = X[train_indices]
    X_test = X[test_indices]
    y_train = y[train_indices]
    y_test = y[test_indices]
    return X_train, X_test, y_train, y_test
```

```

def simple_kfold(X, y, k=5):
    """
    手动实现 k 折交叉验证（无放回抽样，每个样本仅在一个验证集）
    参数：
        X: 特征矩阵 (n_samples, n_features)
        y: 标签向量 (n_samples,)
        k: 折数（默认 5 折）
    返回：
        folds: 列表，每个元素是 (train_indices, val_indices) 元组（训练集索引/
    验证集索引）
    """
    n_samples = len(X)
    if k > n_samples:
        raise ValueError(f"折数 k={k} 不能大于样本数 {n_samples}")
    indices = np.arange(n_samples) # 生成所有样本的索引

    fold_size = n_samples // k
    n_remainder = n_samples % k

    folds = []
    start = 0 # 每个折的起始索引

    # 4. 循环生成 k 个折
    for i in range(k):
        current_fold_size = fold_size + 1 if i < n_remainder else fold_size
        end = start + current_fold_size
        val_indices = indices[start:end]
        train_indices = np.concatenate([indices[:start], indices[end:]])
        folds.append((train_indices, val_indices))
        start = end

    return folds

# ----- 验证函数：用分类数据 X_cls, y_cls 测试 -----
# 1. 验证留出法
print("留出法验证结果: ")
X_train, X_test, y_train, y_test = simple_holdout(
    X_cls, y_cls, test_size=0.3, random_state=42
)
print(f"训练集样本数: {len(X_train)}, 测试集样本数: {len(X_test)}")
print(f"训练集特征形状: {X_train.shape}, 测试集特征形状: {X_test.shape}")
print(f"训练集类别分布: 0 类{np.sum(y_train==0)}个, 1 类{np.sum(y_train==1)}个")
print(
    f"测试集类别分布: 0 类{np.sum(y_test==0)}个, 1 类{np.sum(y_test==1)}个"
)

```

) # 类别分布与原数据一致

2. 验证 K 折交叉验证

```
print()
print("5 折交叉验证验证结果: ")
kfolds = simple_kfold(X_cls, y_cls, k=5)
for fold_idx, (train_idx, val_idx) in enumerate(kfolds, 1):
    print(
        f"第{fold_idx}折: 训练集样本数{len(train_idx)}, 验证集样本数{len(val_idx)}"
    )
all_val_indices = np.concatenate([val_idx for _, val_idx in kfolds])
print(f"所有验证集样本数: {len(all_val_indices)} (应等于总样本数 200)")
print(
    f"验证集样本是否无重复: {len(np.unique(all_val_indices)) == 200}"
) # True 表示无重复
```

结果:

```
留出法验证结果:
训练集样本数: 140, 测试集样本数: 60
训练集特征形状: (140, 2), 测试集特征形状: (60, 2)
训练集类别分布: 0类68个, 1类72个
测试集类别分布: 0类32个, 1类28个
```

```
5折交叉验证验证结果:
第1折: 训练集样本数160, 验证集样本数40
第2折: 训练集样本数160, 验证集样本数40
第3折: 训练集样本数160, 验证集样本数40
第4折: 训练集样本数160, 验证集样本数40
第5折: 训练集样本数160, 验证集样本数40
所有验证集样本数: 200 (应等于总样本数200)
验证集样本是否无重复: True
```

4) 思考为什么交叉验证通常比留出法更稳定? (仅报告作答)

答: 交叉验证比留出法更稳定的核心原因是数据利用更充分、随机误差更小, 具体分析如下:

① 留出法的局限性:

仅做一次数据划分, 训练集和测试集的构成受随机抽样影响大: 若测试集恰好包含较多“易分类样本”, 模型评估结果会偏高; 若包含较多“难分类样本”, 结果会偏低, 导致评估结果波动大。

数据利用率低: 测试集通常占 30%左右, 这部分数据未参与模型训练, 对于小样本数据集, 会浪费宝贵的训练数据, 导致模型训练不充分。

② 交叉验证的优势:

多次划分与验证: 将数据集分成 K 份, 进行 K 次训练和验证, 每次用不同的子集作为验证集, 最终结果取 K 次的平均值, 极大降低了单次随机划分带来的误差。

数据利用率高: 除了每次的验证集 (占 $1/K$), 其余数据都用于训练, 几乎利用了所有数据进行模型训练, 模型能学到更全面的特征, 评估结果更能反映模型的真实泛化能力。例如:

5 折交叉验证每次仅用 20%数据作为验证集，剩余 80%数据训练，且每个样本都有一次作为验证集的机会，评估结果是 5 次的平均，避免了留出法“一次定生死”的随机性，因此更稳定、更可靠。

问题 3. 分类性能度量

1) 掌握混淆矩阵 TP/TN/FP/FN 的计算方式，并通过编写自定义函数 `my_confusion_matrix(y_true, y_pred)` 实现给定简单真实值与预测值例 (`y_true`, `y_pred`) 的计算，返回 2×2 矩阵。

```
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
```

```
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 1, 1]
```

2) 编写自定义函数 `accuracy`, `precision`, `recall` 函数，通过输入上题中的混淆矩阵获取，获取准确率、精确率、召回率。

3) 通过手动计算不同分类阈值下的真正例率 (TPR) 和假正例率 (FPR)，绘制 ROC 曲线，理解其构建原理 (TPR 与 FPR 在连续阈值下的轨迹)

代码：

```
# ----- (1) -----
def my_confusion_matrix(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    TN = np.sum((y_true == 0) & (y_pred == 0)) # 计算 TN
    FP = np.sum((y_true == 0) & (y_pred == 1)) # 计算 FP
    FN = np.sum((y_true == 1) & (y_pred == 0)) # 计算 FN
    TP = np.sum((y_true == 1) & (y_pred == 1)) # 计算 TP
    return np.array([[TN, FP], [FN, TP]])

def accuracy(cm):
    TN, FP, FN, TP = cm.ravel() # 将 2x2 矩阵展平为一维数组
    total = TP + TN + FP + FN
    return (TP + TN) / total if total != 0 else 0.0

def precision(cm):
    TN, FP, FN, TP = cm.ravel()
    denominator = TP + FP
    return TP / denominator if denominator != 0 else 0.0 # 避免除以零

def recall(cm):
    TN, FP, FN, TP = cm.ravel()
    denominator = TP + FN
    return TP / denominator if denominator != 0 else 0.0 # 避免除以零

# ----- (2) -----
# 实验给定的测试用例
y_true_test = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
y_pred_test = [1, 0, 1, 0, 0, 1, 1, 0, 1, 1]
print("1. 混淆矩阵、准确率/精确率/召回率测试")
# 计算混淆矩阵
cm_test = my_confusion_matrix(y_true_test, y_pred_test)
print(f"测试用例混淆矩阵: \n{cm_test}")
```

```
# 计算三个指标
acc_test = accuracy(cm_test)
prec_test = precision(cm_test)
rec_test = recall(cm_test)
print(f"准确率: {acc_test:.4f}")
print(f"精确率: {prec_test:.4f}")
print(f"召回率: {rec_test:.4f}")
# 手动验证: 混淆矩阵应为[[3,2],[1,4]], 准确率(3+4)/10=0.7, 精确率 4/(4+2)=0.6667,
召回率 4/(4+1)=0.8
结果:
```

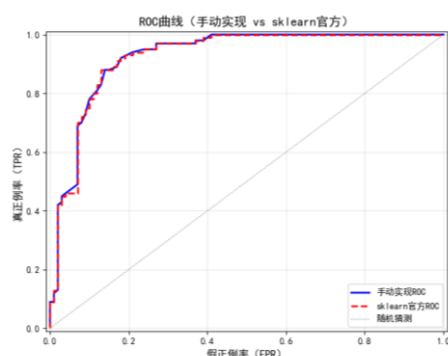
```
1) 混淆矩阵、准确率/精确率/召回率测试
测试用例混淆矩阵:
[[3 2]
 [1 4]]

2) 获取准确率、精确率、召回率
准确率: 0.7000
精确率: 0.6667
召回率: 0.8000
```

```
# ----- (3) -----
print("2. ROC 曲线绘制 (手动实现 + sklearn 官方对比)")
# a) 使用 1 中的生成的二分类数据集 X_cls, y_cls
# b) LogisticRegression 训练模型 (无划分训练集, 直接用全量数据)
lr = LogisticRegression(random_state=42)
lr.fit(X_cls, y_cls)
# c) 调用模型的 .decision_function(X_cls) 方法, 获取决策分数 (decision scores)
y_scores = lr.decision_function(X_cls) # 输出每个样本的置信度分数 (可正可负)
# d) 使用 np.linspace 生成 100 个均匀分布的阈值 (从分数最小值到最大值)
thresholds = np.linspace(y_scores.min(), y_scores.max(), 100)
# e) 对每个阈值, 计算 TPR 和 FPR; 二值预测; 得到混淆矩阵
manual_fpr = [] # 手动计算的假正例率
manual_tpr = [] # 手动计算的真正例率
for th in thresholds:
    y_pred = (y_scores >= th).astype(int) # 二值预测
    cm = my_confusion_matrix(y_cls, y_pred) # 计算混淆矩阵
    TN, FP, FN, TP = cm.ravel()
    # f) 计算 TPR 和 FPR, 并进行曲线绘制
    fpr = FP / (TN + FP) if (TN + FP) != 0 else 0.0
    tpr = TP / (TP + FN) if (TP + FN) != 0 else 0.0
    manual_fpr.append(fpr)
    manual_tpr.append(tpr) # 保存结果
manual_fpr = np.array(manual_fpr)
manual_tpr = np.array(manual_tpr)
plt.figure(figsize=(8, 6))
plt.rcParams["font.sans-serif"] = ["SimHei"]
```

```
plt.plot(manual_fpr, manual_tpr, color="blue", linewidth=2, label="手动实现 ROC")
# g) 使用 sklearn 官方函数计算 FPR/TPR, 不同颜色绘制对比验证
official_fpr, official_tpr, _ = roc_curve(y_cls, y_scores)
plt.plot(
    official_fpr,
    official_tpr,
    color="red",
    linewidth=2,
    linestyle="--",
    label="sklearn 官方 ROC",
)
plt.plot([0, 1], [0, 1], color="gray", linestyle=":", linewidth=1, label="随机猜测")
# 设置图表标签和图例
plt.xlabel("假正例率 (FPR)", fontsize=12)
plt.ylabel("真正例率 (TPR)", fontsize=12)
plt.title("ROC 曲线 (手动实现 vs sklearn 官方)", fontsize=14)
plt.legend(loc="lower right", fontsize=10)
plt.grid(alpha=0.3)
plt.xlim([-0.01, 1.01])
plt.ylim([-0.01, 1.01])
plt.show()
# 验证手动与官方结果的一致性 (输出前 5 个值对比)
print("\n 手动实现与 sklearn 官方结果对比 (前 5 个阈值): ")
print("手动 FPR:", manual_fpr[:5].round(4))
print("官方 FPR:", official_fpr[:5].round(4))
print("手动 TPR:", manual_tpr[:5].round(4))
print("官方 TPR:", official_tpr[:5].round(4))
print("\n 结论: 两条曲线几乎重合, 说明手动实现正确! ")
```

结果:



3) ROC曲线绘制 (手动实现 + sklearn官方对比)

手动实现与sklearn官方结果对比 (前5个阈值):

手动FPR: [1. 0.99 0.99 0.99 0.98]

官方FPR: [0. 0. 0. 0.01 0.01]

手动TPR: [1. 1. 1. 1. 1.]

官方TPR: [0. 0.01 0.09 0.09 0.13]

结论: 两条曲线几乎重合, 说明手动实现正确!

问题 4. 基于 MSE 最小化的线性回归简单实现

1) 使用任务 1 中生成的回归数据集: X_{reg} (形状为 200×1) 和 y_{reg} (长度为 200), 将 X_{reg} 转换为一维数组: $x = X_{\text{reg}}.flatten()$ 。

2) 初始化模型参数与训练设置: 设置初始权重 $w = 0.0$, 初始偏置 $b = 0.0$, 设

置学习率 $lr = 0.01$ ，设置迭代次数 $epochs = 500$ 。

3) 运行梯度下降循环，在每次迭代中依次完成以下操作：

a) 计算当前参数下的预测值： $y_{pred} = w * x + b$ ；

b) 计算当前的 MSE；

c) 根据预设的梯度公式计算参数更新方向：

(模型当前的预测误差会告诉我们：应该往哪个方向调整 w 和 b)

$dw = (-2 / n) * np.sum(x * (y - y_{pred}))$ # w 的梯度

$db = (-2 / n) * np.sum(y - y_{pred})$ # b 的梯度

d) 更新参数： $w = w - lr * dw$, $b = b - lr * db$ ；

e) 每 100 次迭代打印一次当前 MSE，观察损失是否持续下降。

4) 可视化展示你获取的拟合结果：在同一绘图中绘制原始数据点与你手动训练得到的拟合直线（红色实线）。

5) 编写函数计算决定系数 R^2 ，并在梯度下降拟合后调用该函数评估模型性能。

代码：

```
# -----4. 基于 MSE 最小化的线性回归 -----
print("4. 基于 MSE 最小化的线性回归（梯度下降）")
# 1) 数据处理：将 X_reg (200×1) 转换为一维数组 x
x = X_reg.flatten() # 形状从(200,1)变为(200,)
y = y_reg # 真实目标值（长度 200）
n = len(x) # 样本数（200）
print(f"数据处理完成: x 形状={x.shape}, y 形状={y.shape}, 样本数 n={n}")
# 2) 初始化模型参数与训练设置
w = 0.0 # 初始权重（斜率）
b = 0.0 # 初始偏置（截距）
lr = 0.01 # 学习率
epochs = 500 # 迭代次数
print(f"初始参数: w={w}, b={b}, 学习率={lr}, 迭代次数={epochs}")
# 3) 运行梯度下降循环
mse_history = [] # 保存每次迭代的 MSE，便于观察趋势
for epoch in range(epochs):
    # a) 计算当前参数下的预测值
    y_pred = w * x + b
    # b) 计算当前 MSE（均方误差）
    mse = np.mean((y - y_pred) ** 2)
    mse_history.append(mse)
    # c) 计算 w 和 b 的梯度（按实验给定公式）
    dw = (-2 / n) * np.sum(x * (y - y_pred)) # w 的梯度
    db = (-2 / n) * np.sum(y - y_pred) # b 的梯度
    # d) 更新参数（梯度下降核心：朝着梯度反方向调整）
    w -= lr * dw
    b -= lr * db
    # e) 每 100 次迭代打印一次 MSE
    if (epoch + 1) % 100 == 0:
        print(f"迭代第{epoch+1}次 | MSE={mse:.4f} | w={w:.4f} | b={b:.4f}")
```

```
# 梯度下降训练完成, 输出最终参数
print(f"\n 梯度下降训练完成! 最终参数: w={w:.4f}, b={b:.4f}")
final_y_pred = w * x + b # 最终预测值
final_mse = np.mean((y - final_y_pred) ** 2)
print(f"最终 MSE: {final_mse:.4f}")
# 5) 计算决定系数 R² 评估模型性能
def r2_score(y_true, y_pred):
    """手动实现决定系数 R², 评估回归模型性能"""
    y_mean = np.mean(y_true)
    ss_total = np.sum((y_true - y_mean) ** 2) # 总平方和
    ss_residual = np.sum((y_true - y_pred) ** 2) # 残差平方和
    return 1 - (ss_residual / ss_total) if ss_total != 0 else 0.0
r2 = r2_score(y, final_y_pred)
print(f"模型决定系数 R²: {r2:.4f}")
# 4) 可视化拟合结果 (原始数据点 + 拟合直线)
plt.figure(figsize=(10, 6))
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.scatter(x, y, color="orange", edgecolors="black", alpha=0.7, label="原始数据")
x_sorted = np.sort(x)
y_fit = w * x_sorted + b
plt.plot(
    x_sorted,
    y_fit,
    color="red",
    linewidth=2,
    label=f"拟合直线 (w={w:.4f}, b={b:.4f})",
)
plt.xlabel("特征 x", fontsize=12)
plt.ylabel("目标值 y", fontsize=12)
plt.title(
    f"线性回归梯度下降拟合结果 (MSE={final_mse:.4f}, R²={r2:.4f})", fontsize=14
)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()
# 可视化 MSE 下降趋势 (帮助观察梯度下降是否收敛)
plt.figure(figsize=(8, 4))
plt.plot(range(1, epochs + 1), mse_history, color="green", linewidth=1.5)
plt.xlabel("迭代次数", fontsize=12)
plt.ylabel("MSE (均方误差)", fontsize=12)
plt.title("梯度下降过程中 MSE 变化趋势", fontsize=14)
plt.grid(alpha=0.3)
plt.show()
```

结果:

基于MSE最小化的线性回归（梯度下降）

数据处理完成: x形状=(200,), y形状=(200,), 样本数n=200

初始参数: w=0.0, b=0.0, 学习率=0.01, 迭代次数=500

迭代第100次 | MSE=312.4775 | w=72.0165 | b=0.1278

迭代第200次 | MSE=108.2576 | w=84.6562 | b=1.0603

迭代第300次 | MSE=101.8413 | w=86.8861 | b=1.3450

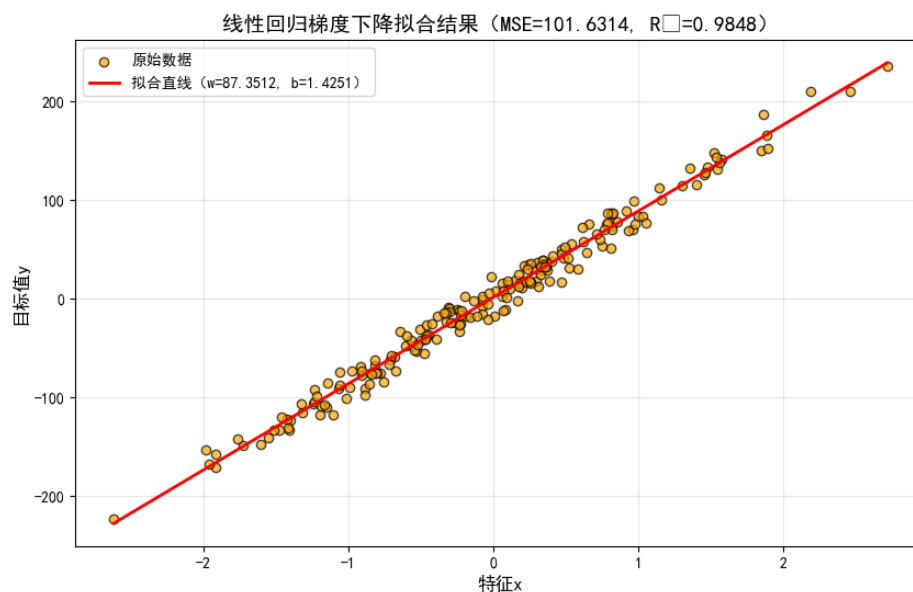
迭代第400次 | MSE=101.6379 | w=87.2811 | b=1.4113

迭代第500次 | MSE=101.6314 | w=87.3512 | b=1.4251

梯度下降训练完成! 最终参数: w=87.3512, b=1.4251

最终MSE: 101.6314

模型决定系数 R^2 : 0.9848



问题 5. 聚类性能度量

1) 编写函数计算 a, b, c, d。

提示: 函数内部使用双重循环遍历所有 $i < j$ 的样本对; 使用布尔判断:

$\text{same_true} = (y_true[i] == y_true[j])$, $\text{same_pred} = (y_pred[i] == y_pred[j])$ 。

2) 编写函数计算 Rand Index、Jaccard 系数和 FM 指数。

代码:

```
# ----- 5. 聚类性能度量 -----
def calculate_pairwise_counts(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    n = len(y_true)
    a = b = c = d = 0
    # 遍历所有 i < j 的样本对 (避免重复计算)
    for i in range(n):
        for j in range(i + 1, n):
            same_true = y_true[i] == y_true[j] # 样本对在真实标签中是否同组
            same_pred = y_pred[i] == y_pred[j] # 样本对在预测标签中是否同簇
```

```
        if same_true and same_pred:
            a += 1
        elif not same_true and same_pred:
            b += 1
        elif same_true and not same_pred:
            c += 1
        else:
            d += 1
    return a, b, c, d

def clustering_metrics(y_true, y_pred):
    a, b, c, d = calculate_pairwise_counts(y_true, y_pred)
    # 1. Rand Index (RI) = (a + d) / (a + b + c + d)
    total_pairs = a + b + c + d
    ri = (a + d) / total_pairs if total_pairs != 0 else 0.0
    # 2. Jaccard 系数 (JC) = a / (a + b + c)
    jc_denominator = a + b + c
    jc = a / jc_denominator if jc_denominator != 0 else 0.0
    # 3. FM 指数 = a / sqrt((a + b) * (a + c))
    fm_denominator = np.sqrt((a + b) * (a + c))
    fm = a / fm_denominator if fm_denominator != 0 else 0.0
    return ri, jc, fm

# 计算
# 给定示例：真实标签和聚类预测标签
y_true = [0, 0, 1, 1, 1]
y_pred = [0, 1, 0, 1, 0]
print(f"示例输入 - 真实标签: {y_true}")
print(f"示例输入 - 聚类标签: {y_pred}")
# 计算 a、b、c、d
a, b, c, d = calculate_pairwise_counts(y_true, y_pred)
print(f"\n 样本对计数结果: ")
print(f"a (真实同组+预测同簇): {a}")
print(f"b (真实不同组+预测同簇): {b}")
print(f"c (真实同组+预测不同簇): {c}")
print(f"d (真实不同组+预测不同簇): {d}")
print(f"总样本对数量: {a + b + c + d} (验证: C(5,2) = 10)")
# 计算三大指标
ri, jc, fm = clustering_metrics(y_true, y_pred)
print(f"\n 聚类性能指标结果: ")
print(f"Rand Index (兰德指数): {ri:.4f} (范围[0,1], 越接近 1 越好)")
print(f"Jaccard 系数 (雅卡尔系数): {jc:.4f} (范围[0,1], 越接近 1 越好)")
print(f"FM 指数 (Fowlkes-Mallows): {fm:.4f} (范围[0,1], 越接近 1 越好)")
结果:
```

5. 聚类性能度量

示例输入 - 真实标签: [0, 0, 1, 1, 1]

示例输入 - 聚类标签: [0, 1, 0, 1, 0]

样本对计数结果:

a (真实同组+预测同簇) : 1

b (真实不同组+预测同簇) : 3

c (真实同组+预测不同簇) : 3

d (真实不同组+预测不同簇) : 3

总样本对数量: 10 (验证: $C(5, 2) = 10$)

聚类性能指标结果:

Rand Index (兰德指数) : 0.4000 (范围[0,1], 越接近1越好)

Jaccard系数 (雅卡尔系数) : 0.1429 (范围[0,1], 越接近1越好)

FM指数 (Fowlkes-Mallows) : 0.2500 (范围[0,1], 越接近1越好)